

Exercise Sheet 5

Lab Sessions: May 28/29, 2026

1 Preparation

For this exercise, we provide files in the [RepEng FIMGit Repository](#) (directory `LabSession5/`). Update your local repository that you cloned in the first lab session:

1. Open a terminal in your local repository.
2. Pull the latest changes using `git pull`.

2 Basic Git

In this exercise we will review basic concepts of Git. It is assumed you worked through the [Udacity Git Course](#)¹ prior to the lab session.

Copy the folder `RepEng/LabSession5` to `MyLabSession5`:

Terminal Session

bash

```
user@host$ cp -r RepEng/LabSession5 MyLabSession5
user@host$ cd MyLabSession5
```

Then carry out the following tasks.

- (a) Initialize a new Git repository.
- (b) Add all files inside the folder to the staging index.
- (c) Make an initial commit.
- (d) Assign the tag “LabSession5-v1” to the latest commit.
- (e) Create a new branch called “MyLabSession5” and check it out.
- (f) Change the output of the shell script to “Hello from Lab Session 5”.
- (g) Check the status of the repository. You should observe that `hello.sh` is listed as modified but not staged for commit.
- (h) Add the changed file to the staging index.

¹<https://www.udacity.com/course/version-control-with-git--ud123>

- (i) Check the status of the repository. You should observe that `hello.sh` is listed as modified and staged for commit.
- (j) Make a commit.
- (k) Create a new file with name `todos.txt`.

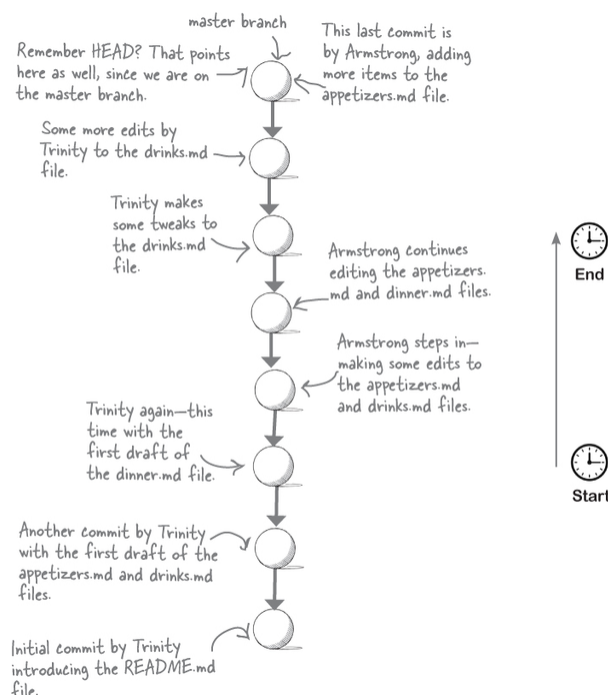
```
| user@host$ touch todos.txt
```

Make sure the new file is ignored by Git.

- (l) Check the status of the repository. You should observe that the file `.gitignore` is listed as untracked file but not the file `todos.txt`, which is ignored by Git.

3 Tracking Contributions

The following material is an excerpt from the book “Head first git”, by Raju Gandhi. The repository is available on [GitHub²](#).



- (a) Clone and enter the repository.
- (b) Call `git blame drinks.md`. How do you read the output?
- (c) You are going to use `git blame` to inspect the revisions that have been made to the `appetizers.md` file in the `gitanjali-aref-wedding-plans` repository.

Answer the following questions.

- (i) How many authors have contributed to this file?
- (ii) When was the last edit to this file made?
- (iii) Who last edited line number 5?

²<https://github.com/looselytyped/gitanjali-aref-wedding-plans>

4 Rewriting Histories

A clear Git history documents how a project evolved over time. In this exercise, we rewrite an existing git history.

- (a) Clone and enter the example [ReproEng GitHub Repository](https://github.com/ReproEng/LabSession2):

```
user@host$ git clone https://github.com/ReproEng/LabSession2
user@host$ cd LabSession2
```

- (b) Inspect the changes in branch `master` using `git log`. Decide which changes should be preserved for reproducers, and which ones fix honest mistakes that are not of interest in the future.
- (c) Perform an *interactive rebase* to merge the commits into a smaller set of readable commits:

```
user@host$ git rebase -i <commit>
```

Hints:

- Create a separate branch for your rebasing experiments to avoid having to re-clone the repository.
- `git reflog` can be helpful when you have made a mistake; it lists all previous states known to Git. In case you make a mistake when merging commits, you can reset to a previous good state using

```
user@host$ git reset --hard <commit>
```

- (d) Check out a new branch based on your current results. Merge the changes from branch `i18n` (internationalizing the software) and resolve any arising merge conflicts.
- (e) Create a new branch with a linear history (free of merges) that includes the (updated) changes from branch `i18n` in a separate commit, starting from the branch created with the interactive rebase in (c).
- (f) Review the content of all commits from the previous step. Change the commit message of commits where the message does not reflect the content of the commit anymore (use the *reword* feature of interactive rebasing).

5 Rewriting History: Working with Patches

Sometimes it is necessary to modify existing code, e.g., from a third-party repository. Diff patches make these changes transparent. We will create a Docker container that applies a patch to a C-program `hello.c`.

- (a) Enter the folder `RepEng/LabSession5/`.
- (b) Create a copy of `hello.c` with file name `hello_new.c`. Modify this copy such that “Hello World” is printed.
- (c) Create a diff patch containing the changes made in (b):

```
| user@host$ diff -u hello.c hello_new.c > hello.patch
```

Then edit the `Dockerfile` such that it applies the patch and compiles the patched program.

Hints:

- Patches can be applied using the command `patch <file> <patch_file>`.
- You can compile a C-program using the command `gcc -o <output_file> <source_file>`.

- (d) Build and run the docker container and run the compiled program.

6 Git (Antwort-Wahl-Verfahren)

In this exercise, each question has exactly one correct answer. Mark the correct answer.

Git Logs

Consider the following terminal session.

Terminal Session

bash

```
user@host$ git log --format=fuller --stat
commit f8c3a91d4b7e2c6a5d0f1e9b8a3c2d1e0f9a8765 (HEAD -> main)
Author:      Alice Miller <alice@example.com>
AuthorDate:  Tue May 12 14:18:09 2026 +0200
Commit:      Chris Admin <chris@example.com>
CommitDate:  Tue May 12 14:27:41 2026 +0200
```

Fix typo in installation instructions

```
README.md | 4 ++--
1 file changed, 2 insertions(+), 2 deletions(-)
```

```
commit c41b72e9a6d3f0c8b5e1a2d4f9c7b6a0e3d2c145
Author:      Ben Carter <ben@example.com>
AuthorDate:  Mon May 11 09:42:16 2026 +0200
Commit:      Ben Carter <ben@example.com>
CommitDate:  Mon May 11 09:45:03 2026 +0200
```

Add input validation for registration form

```
src/register.js | 18 ++++++-----
tests/form.test.js | 12 ++++++
2 files changed, 25 insertions(+), 5 deletions(-)
```

```
commit a93d5f2b0c8e7a6d4f1b9c3e2a0d5b7c6e8f1234
Author:      Dana Smith <dana@example.com>
AuthorDate:  Fri May 8 16:03:55 2026 +0200
Commit:      Alice Miller <alice@example.com>
CommitDate:  Fri May 8 17:12:10 2026 +0200
```

Add user profile and registration pages

```
src/profile.html | 36 ++++++
src/register.js | 24 ++++++
src/style.css | 14 ++++++
3 files changed, 74 insertions(+)
```

```
commit b7e4c19d2a8f6c0e5b3d1a9f7c2e4b6a0d8c1357
Author:      Dana Smith <dana@example.com>
AuthorDate:  Thu May 7 11:21:30 2026 +0200
Commit:      Dana Smith <dana@example.com>
CommitDate:  Thu May 7 11:21:30 2026 +0200
```

Initial project structure

```
README.md | 20 ++++++
index.html | 18 ++++++
app.js | 7 ++++++
3 files changed, 45 insertions(+)
```

- (a) Who authored the latest code changes?
- ☐ Alice Miller ☐ Chris Admin ☐ Dana Smith ☐ None of these options
- (b) How many different persons authored code changes?
- ☐ 1 ☐ 2 ☐ 3 ☐ 4
- (c) How many commits delete a file?
- ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4
- (d) Which file has undergone the most changes in total, measured in lines of code?
- ☐ README.md ☐ index.html ☐ app.js
- ☐ src/profile.html ☐ src/register.js ☐ None of these options

Git Diffs

Consider the following terminal session³.

```

Terminal Session bash
user@host$ git diff

diff --git a/resume.md b/resume.md
index fef1862..4f4ea8c 100644
--- a/resume.md
+++ b/resume.md
@@ -5,7 +5,7 @@ To leverage my language skills as Chief Communications Officer

  ## Languages
  English (native)
  -Romulan (beginner)
  +Klingon (fluent)

  ## Education
  Starfleet Academy, San Francisco, CA
@@ -14,4 +14,4 @@ Valedictorian, Class of 2018
  ## Experience
  2018-2020, Federation Starship Atlantis - Communications Ensign

  -Routed incoming subspace communications to the correct officer
  \ No newline at end of file
  +Routed all incoming subspace communications to the correct officer
  \ No newline at end of file

```

- (e) How many of the following lines are present in the latest version of `resume.md` starting with line 5 in that file?
- `## Languages`
 - `English (native)`
 - `Romulan (beginner)`
 - `Klingon (fluent)`
- ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

³Source: Raju Gandhi: *Head First Git*, O'Reilly

(f) Next, you apply the following commands:

```
user@host$ git add .
user@host$ git commit -m "Edit resume.md"
```

Then you check the log of the Git repository:

Terminal Session	bash
<pre>user@host\$ git log commit fd3f2aa4775bdf88bcd4a80b06fc808ab38a1083 (HEAD -> main) Author: testuser <testuser@example.com> Date: Tue May 26 23:37:04 2026 +0200 Edit resume.md commit 9004a6b5755bbfa1d586aaf0c56aa106ba8f80fa (other-branch) Author: testuser <testuser@example.com> Date: Tue May 26 23:33:11 2026 +0200 Initial commit</pre>	

You checkout another branch:

```
user@host$ git checkout other-branch
```

How many of the following lines are currently present in `resume.md` starting with line 5 in that file?

- 2018-2020, Federation Starship Atlantis - Communications Ensign
- Routed incoming subspace communications to the correct officer
- Routed all incoming subspace communications to the correct officer
- Klingon (fluent)

☐ 0

☐ 1

☐ 2

☐ 3

☐ 4